

Auditoría SatsPath — Qué Está Implementado, Qué Falta, y Plan de Ramas

Fecha: Julio 2026

Rama actual: feat/satspath-receiver-flow (5 commits adelante de main)

Repo: Truja503/satspath

Wallet: /home/chelo/antigravity/PlanB/satspath/wallet (Arkade Money)

1. Estado de Ramas (El “Desvergue”)

Diagnóstico

El repo tiene **18 ramas remotas** y **3 locales**. La main tiene todo el código Rust compilado con 277 tests. La rama actual (feat/satspath-receiver-flow) trajo la wallet completa al monorepo. Hay un **master** viejo que es un ancestro totalmente distinto.

Ramas YA MERGEADAS en main (seguras de eliminar)

Estas ramas ya están contenidas en main — su trabajo ya está ahí:

Rama	Contenido	Acción
origin/chore/remove-stray-patches	Limpiar parches	Borrar
origin/codex/p2p-daemon	Resolución P2P	Borrar
origin/codex/p2p-publish	Reparar P2P publish	Borrar
origin/feat/bip353-dns	BIP-353 DNS	Borrar
origin/feat/holepunch-p2p-sdk	Holepunch P2P SDK	Borrar
origin/feat/peer-export	Export/Import perfiles	Borrar
origin/feat/quote-json	Control response JSON	Borrar
origin/feat/swap-engine	Swap bridge Ark	Borrar
origin/feat/wallet-profile	Wallet manager	Borrar
origin/feature/ark-send	Ark send/recv	Borrar
origin/feature/payment	Outros hipotesis	Borrar
origin/feature/protocol	Intercambio de pruebas	Borrar
origin/feature/protocol	Intercambio de pruebas	Borrar
origin/fix/v0-priority	Fix de prioridad	Borrar
origin/feature/mainnet	Plan de mainnet v2	Borrar
origin/feature/mainnet	LN-test mainnet	Borrar (ancestro viejo)

Ramas NO mergeadas (tienen trabajo único)

Rama	Commits extra	Contenido	Acción
origin/feat/satspath-receiver-flow	5	Wallet + docs arquitectura	MERGEAR a main Revisar: basada en master viejo, probablemente incompatible Superseded por feat/satspath-receiver-flow → Borrar Borrar — todo migró a main
origin/feat/ark5-vtxo-verification-integration	5	Verificación VTXO Ark	
origin/feat/ark4-wallet-integration	4	Wallet integration vieja	
origin/master	—	Ancestro original (Initial commit)	

Plan de Limpieza (Comandos)

```
# 1. Mergear la rama actual a main
git checkout main
git merge feat/satspath-receiver-flow --no-ff -m "feat: merge wallet + receiver flow into main"

# 2. Borrar ramas ya mergeadas (remotas)
for b in \
  chore/remove-stray-scratch-files \
  codex/p2p-daemon-resolver \
  codex/p2p-publish-repair \
  feat/bip353-dns-resolution \
  feat/holepunch-p2p-sdk \
  feat/peer-export-import \
  feat/quote-json-contract \
  feat/swap-engine-ark-bridge \
  feat/wallet-profile-manager \
  feature/ark-send-receive-swaps \
  feature/payment-method-ownership-proofs \
  feature/protocol-invite-routing-model \
  fix/v0-priority-issues \
  feature/mainnet-preview-mode-v2 \
  feature/mainnet-ln-test \
  feat/arkade-wallet-integration \
  master; do
  git push origin --delete "$b"
done

# 3. La rama feat/ark-vtxo-verification-integration tiene código interesante
# pero está basada en el master viejo. Revisar si vale cherry-pickear algo.
# Si no, borrar también.

# 4. Limpiar ramas locales
git branch -d feat/satspath-receiver-flow fix/v0-priority-issues feat/swap-engine-ark-bridge checkout 2

# 5. Limpiar referencias locales obsoletas
git remote prune origin

[!IMPORTANT] Después de esto, el repo quedará con solo main y opcionalmente feat/ark-vtxo-verification-integration si decides cherry-pick.
```

2. Qué SÍ está implementado (verificado en código)

Core del Protocolo — satspath-core

Feature	Archivo(s)	Estado
PaymentProfile con Lightning, Onchain, Ark	profile.rs	Completo
SignedPaymentProfile con firma Schnorr	profile.rs	Completo
Generación de keypair secp256k1	crypto.rs	Completo

Feature	Archivo(s)	Estado
Firma con domain separator (<code>SatsPathProfileV1</code>)	crypto.rs	Completo
Verificación de firma Schnorr	crypto.rs	Completo
Verificación de expiración	crypto.rs	Completo
Nonces de replay protection	crypto.rs	Completo
Canonical JSON serialization	crypto.rs	Completo
Validación de perfiles públicos	validation.rs	Completo
Rechazo de material privado (xprv, seeds, etc.)	validation.rs	Completo
Validación de Bitcoin Address + red	validation.rs	Completo
Validación de Lightning Address	validation.rs	Completo
Validación de BOLT12 offer (bech32 lno)	validation.rs	Completo
Invitaciones firmadas	lib.rs	Completo
Verificación de invitaciones	lib.rs	Completo
Key Rotation (tipos + validación)	rotation.rs	Tipos + lógica
Ownership Proofs (firmas + well-known)	ownership.rs	Extenso (73KB)
Privacy (identifier hash, masking)	privacy.rs	Completo
BIP-321 parsing (bitcoin: URI)	bip321.rs	Completo
BIP-353 resolve + publish	bip353.rs, bip353_publish.rs	Completo (20KB + 16KB)
Ark: receive pointer, validation, ownership	ark.rs	Tipos + validación (18KB)
Peer Registry (local P2P)	peer_registry.rs	Completo
Local registry	registry.rs	Completo
Split Payment (tipos)	split.rs	Solo tipos (1.8KB)
Payment Pointer	pointer.rs	Completo
Codec (universal request encoding)	codec.rs	Completo

Resolver Chain — `satspath-core/resolvers`

Resolver	Archivo	Estado
<code>ChainResolver</code> (compositor)	resolver.rs	Con protección anti-sustitución (SEC-02)
<code>HttpResolver</code> (HTTPS well-known + NIP-05)	http.rs	10KB, verifica firma + expiración
<code>Bip353Resolver</code> (DNS TXT)	bip353.rs	Funcional
<code>NostrResolver</code> (NIP-05)	nostr.rs	14KB
<code>PearResolver</code> (Hyperswarm P2P)	pear.rs	Funcional pero usa <code>Command::new("node")</code> —
<code>PlatformResolver</code> (placeholder)	platform.rs	depende de repo paths Solo 494 bytes — stub

Router — `satspath-router`

Feature	Archivo	Estado
<code>select_route()</code> (Lightning → Onchain → Ark)	router.rs	Completo + tests
Fee estimation (mempool.space API)	fees.rs	Funcional
Scoring engine	scoring.rs	14KB
Priority routing	priority.rs	7.8KB
Urgency levels	urgency.rs	Completo
LNURL-pay 2-step (metadata → invoice)	lightning.rs	Completo con validación BOLT11
BOLT12 handling	bolt12.rs	Tipos + parsing (10KB), sin flujo completo
Silent Payments	silent_payments.rs	Tipos + validación (10KB), sin test con wallet real
Split Payments	split_payments.rs	Validación (6.6KB), sin atomicidad ni ejecución
Key Rotation router	key_rotation.rs	Tipos + validación
Ark routes	ark_routes.rs	Planificación (7.7KB), sin test con ASP real
QuoteResponse contract	quote_response.rs	23KB contrato estable
BIP-353 preview	bip353_preview.rs	Funcional

WASM — satspath-wasm

Feature	Archivo	Estado
<code>generate_identity_keypair()</code>	crypto.rs	Funcional
<code>sign_profile_json()</code>	crypto.rs	Funcional
<code>verify_signed_profile()</code>	crypto.rs	Con legacy fallback
<code>quote()</code> (resolve + route)	router.rs	17KB
Resolver chain WASM	resolver.rs	15KB
<code>topic_for_alias()</code>	topic.rs	Funcional
Types + constants	types.rs	9KB

FFI (UniFFI → Kotlin/Swift) — satspath-ffi

Feature	Archivo	Estado
Identity management	identity.rs	Funcional
Profile creation + signing	profile.rs	Funcional
Crypto (verify, fingerprint)	crypto.rs	Funcional
Resolver FFI	resolver.rs	Funcional
Router FFI	router.rs	Funcional
Type conversions	convert.rs	12KB
P2P bridge	p2p.rs	Placeholder — TODO (Phase 5)

Daemon (satspathd) — satspathd

Feature	Estado
HTTP API (health, profile, resolve, quote, pay, send, receive, dns)	Funcional (58KB main.rs)
UI HTML embebida	Funcional (22KB index.html)
QR SVG generation	Funcional

Wallet Integration (Arkade Money) — wallet

Feature	Archivo	Estado
WASM bridge (<code>satspath.ts</code>)	<code>satspath.ts</code>	<code>routePayment()</code> + <code>buildSatsPathProfile()</code>
Receiver UI (Settings → SatsPath Profile)	<code>SatsPathProfile.tsx</code>	Genera keypair + firma + guarda en LocalStorage
Send Form interceptor	<code>Form.tsx</code>	Detecta alias @ → rutea con SatsPath
Storage (LocalStorage)	<code>storage.ts</code>	Save/read profile
LaserBeam animation	<code>LaserBeam.tsx</code>	Efecto visual durante generación

3. Qué NO está implementado (verificado en código)

Crítico para producción

Item	Detalle	Dificultad
Verificación real de identidad	<code>MockEmailVerifier</code> en <code>platform.rs</code> . Magic links, tokens, rate limiting — nada real	Alta
Procedencia del perfil (trust model)	<code>ResolvedProfile</code> con <code>source</code> , <code>trust_level</code> , <code>identifier_verified</code> no existe. El resolver devuelve solo <code>SignedPaymentProfile</code>	Alta
Publicación real del perfil	La wallet genera el JSON firmado pero NO lo publica. Solo “Copy to clipboard”. No hay publish a Nostr, DNS, ni HTTPS	Alta
Seguridad del daemon	Sin auth, sin CSRF, CORS abierto, sin rate limiting, sin separación admin/public	Alta
Protección de clave de identidad	Secret key guardada en LocalStorage en texto plano (wallet) y archivos JSON (daemon). Sin Keychain/Keystore	Alta
SSRF protection	Los resolvers HTTP/LNURL no bloquean localhost/redes privadas	Media
P2P embebido (FFI)	<code>p2p.rs</code> es un placeholder con <code>TODO(Phase 5)</code>	Media

Item	Detalle	Dificultad
WASM .wasm binary	El wallet importa <code>satspath-wasm</code> pero NO hay el archivo <code>satspath_wasm_bg.wasm</code> compilado en <code>public/</code>	Crítico para funcionar

Parcialmente implementado

Item	Estado actual	Qué falta
BOLT12	Validación de formato bech32 (lno) en <code>validation.rs</code> , tipos en <code>bolt12.rs</code>	Flujo completo: <code>offer</code> → <code>invoice request</code> → <code>invoice</code> → verificación → <code>handoff</code>
Silent Payments	Tipos + validación de SP pubkey, campo <code>silent_payment_pubkey</code> en <code>Onchain</code>	Generación BIP-352, derivación ephemeral address, test con wallet real
Split Payments	Solo structs en <code>split.rs</code> (1.8KB), validación en router	Ejecución multi-rail, atomicidad, manejo de fallo parcial
Ark interop	Tipos + validación de pointer/proof, routing a Ark como fallback, <code>ArkadeManual</code> <code>SwapDirective</code>	Test con ASP real (Arkade), formato de receive pointer compatible, flujo Ark-to-Ark
Key Rotation	Tipos + <code>rotate_identity_key()</code> + validación	Integración con resolvers (rechazar replays), propagación a publicaciones
BIP-353 DNSSEC	Resolución DNS funcional con DoH	DNSSEC operativo, publicación automatizada, rotación TTL
PearResolver	Funcional pero spawndea <code>node index.js</code>	Depende de paths relativos del repo — no funciona como app instalada
Send Flow en Wallet	Intercepta @ aliases, rutea, muestra método + fee	No ejecuta el pago vía WASM — solo muestra info. Mock hardcoded para <code>chello@dev.idk</code>

No implementado en absoluto

Item	Detalle
P2P soberano (ECDH, ChaCha20, SYN/SYN-ACK)	Plan en <code>satspath_architecture_wallet.md</code> — 0% código
Web of Trust (NIP-65)	Solo en docs — 0% código
Hashcash anti-spam	Solo en docs — 0% código
Métricas / Observabilidad	Nada implementado
CI obligatorio (gate)	Workflow existe pero no es gate
Releases firmadas	No hay releases
Test vectors públicos	No existen
Auditoría de criptografía	No realizada

4. TODO List — SatsPath → Producción + Integración Wallet

Fase 0: Limpieza (1-2 días)

- ☐ Merge `feat/satspath-receiver-flow` → `main`

- ☐ Borrar las 15+ ramas mergeadas/obsoletas (ver comandos arriba)
 - ☐ Verificar que `cargo test` pasa en main con la wallet incluida
 - ☐ Compilar `satspath-wasm` y colocar `.wasm` en `wallet/public/`
 - ☐ Eliminar el mock hardcoded `chello@dev.idk` de `satspath.ts`
-

Fase 1: MVP Funcional (1-2 semanas)

1.1 WASM Build Pipeline

- ☐ Script/Makefile: `wasm-pack build crates/satspath-wasm --target web --out-dir ../../wallet/public/`
- ☐ Verificar que la wallet carga el WASM y puede generar keypairs
- ☐ Verificar que `quote()` funciona end-to-end en browser

1.2 Publicación del Perfil

- ☐ Implementar `publish` a Nostr (NIP-01 event con perfil firmado) desde el WASM bridge
- ☐ Alternativa: POST al daemon `satspathd` que publique vía Nostr relay
- ☐ Agregar botón “Broadcast to Nostr” en `SatsPathProfile.tsx`

1.3 Resolución Real en Send

- ☐ En `Form.tsx`: cuando `SatsPath` devuelve `RouteResult`:
 - Si es Lightning → usar el payload como Lightning Address y continuar con el flujo existente de LN de la wallet
 - Si es Onchain → construir BIP-21 URI y continuar con flujo onchain existente
 - Si es Ark → manejar con wallet adapter de Arkade

1.4 Trust Model Básico

- ☐ Agregar `ResolvedProfile` wrapper con `source: ResolverSource` al resultado del WASM `quote()`
 - ☐ Mostrar indicador visual en la wallet: “Verificado por DNS” / “Verificado por Nostr” / “Sin verificar”
-

Fase 2: Seguridad Avanzada y Resistencia Cuántica (1-2 semanas)

2.1 Protección de Secret Key

- ☐ En la wallet: cifrar `secretKey` en `LocalStorage` con password del usuario o derivar de Arkade wallet seed
- ☐ En `satspathd`: migrar de JSON plano a Keychain/Secret Service (ya existe code path AES-GCM, verificar que funcione)

2.2 Verificación de Identidad

- ☐ Implementar `EmailVerifier` real con:
 - ☐ Token aleatorio de un solo uso
 - ☐ Expiración (10 min)
 - ☐ Rate limiting
- ☐ Para dominios: verificar ownership via DNS TXT record o HTTPS well-known

2.3 Daemon Hardening

- ☐ Auth token para API admin
- ☐ CORS restrictivo
- ☐ SSRF: bloquear resolvers a localhost/private nets para contenido remoto

- ☐ Rate limiting
- ☐ Max payload size

2.4 Criptografía Post-Cuántica (PQC) y Semi-Cuántica

- ☐ **Esquemas Híbridos de Firma:** Añadir soporte para firmas híbridas combinando ECDSA/Schnorr tradicional con **ML-DSA (Dilithium)**. Esto asegura compatibilidad actual mientras protege las identidades contra futuros ataques de computación cuántica (Shor's algorithm).
 - ☐ **Intercambio de Claves P2P:** Transicionar el P2P ECDH (Fase 5) a un KEM híbrido que combine curvas elípticas (X25519) con **ML-KEM (Kyber)** para establecer los secretos compartidos.
 - ☐ **Migración de Perfiles:** Definir un field en el JSON del profile para anunciar soporte PQC y transicionar perfiles clásicos a PQC de forma fluida.
-

Fase 3: Integración Completa con Arkade Wallet (2-3 semanas)

3.1 Receiver Flow Completo

- ☐ Crear perfil → Firmar → Publicar a Nostr/DNS → Confirmar publicación
- ☐ Mostrar QR del perfil firmado
- ☐ Permitir actualización del perfil (increment sequence)
- ☐ Manejar expiración y renovación automática

3.2 Sender Flow Completo

- ☐ Resolve → Verify → Quote → Confirm → Execute Payment
- ☐ Lightning: obtener invoice BOLT11 vía LNURL → pagar con wallet LN
- ☐ Onchain: construir tx con BIP-21 → firmar con wallet onchain
- ☐ Ark: construir intent → enviar vía Arkade SDK

3.3 Wallet Adapter

- ☐ Interfaz `WalletAdapter` en el WASM bridge que reporte:
 - ☐ Balance por rail (LN/onchain/Ark)
 - ☐ Capacidad de pago
 - ☐ Resultado del pago (éxito/fallo)
- ☐ Integrar con providers existentes de Arkade wallet (`WalletContext`, `svcWallet`)

3.4 Fallback entre Rails

- ☐ Si Lightning falla → intentar Ark → intentar Onchain
 - ☐ Mostrar al usuario el fallback con razón legible
-

Fase 4: Rails Avanzados (3-4 semanas)

- ☐ **Ark interop real:** probar con ASP de Arkade (formato de pointer, URI, QR)
 - ☐ **BOLT12:** implementar flujo completo offer → invoice con wallet LDK
 - ☐ **Silent Payments:** generación BIP-352 + derivación ephemeral address
 - ☐ **Split Payments:** semántica definida (multi-receiver vs multi-rail) + ejecución
-

Fase 5: P2P Soberano (4-6 semanas)

- ☐ Derivación de tópico seguro (no alias en texto plano)
 - ☐ ECDH compartido para P2P cifrado
 - ☐ Protocolo SYN/SYN-ACK para pagos P2P
 - ☐ Web of Trust con NIP-65
 - ☐ Hashcash anti-spam
 - ☐ Migrar PearResolver de `Command::new("node")` a transport embebido
-

Fase 6: Producción (2-3 semanas)

- ☐ CI como gate obligatorio
 - ☐ Releases versionadas + changelog
 - ☐ Test vectors públicos
 - ☐ Auditoría de criptografía externa
 - ☐ Fuzzing de perfiles y URIs
 - ☐ Documentación API estable
-

5. Resumen Rápido

SATSPATH STATUS

Core (crypto + profiles + validation)
95%

Resolver Chain
80%

Router + Scoring
80%

WASM Bindings
85%

FFI (Kotlin/Swift)
60%

Wallet Integration
40%

Publish + Resolve E2E
20%

Security Hardening
15%

P2P Soberano
5%

[!TIP] **Prioridad inmediata:** Limpiar ramas → compilar WASM → que la wallet pueda

hacer el flujo Receiver: crear perfil + publicar y Sender: resolver + quote + mostrar confirmación de verdad. Todo lo demás viene después.